📝 Exercices - Gestion des Exceptions

Exercice 1: Conversion de types basique

Convertir une chaîne en nombre

Facile

Écrivez une fonction `str_to_int(text)` qui :

- Prend une chaîne de caractères en paramètre
- Essaie de la convertir en entier
- Si la conversion échoue, affiche un message d'erreur et retourne 0
- Si la conversion réussit, retourne l'entier

```
def str_to_int(text): # À vous de jouer ! pass # Tests print(str_to_int("42")) # Attendu: 42
print(str_to_int("abc")) # Attendu: 0 + message d'erreur print(str_to_int("-15")) # Attendu: -15
```

💡 **Indice:** Utilisez un bloc try-except pour capturer l'exception ValueError.

Exercice 2: Accès à une liste

Récupérer un élément de liste en toute sécurité

Facile

Écrivez une fonction `safe_list_access(lst, index)` qui :

- Tente d'accéder à l'élément à l'index donné
- Si l'index est hors limites, affiche un message et retourne None
- Sinon, retourne l'élément

```
def safe_list_access(lst, index): # À vous de jouer ! pass # Tests numbers = [10, 20, 30, 40]
print(safe_list_access(numbers, 0)) # Attendu: 10 print(safe_list_access(numbers, 10)) # Attendu: None +
message
```

💡 **Indice:** C'est l'exception IndexError qui se lève quand on accède à un mauvais index.

Exercice 3: Gestion de fichier

Lire un fichier en toute sécurité

Moyen

Écrivez une fonction `read_file(filename)` qui :

- Ouvre et lit le contenu d'un fichier
- Capture `FileNotFoundError` si le fichier n'existe pas
- Capture `IOError` pour les autres problèmes de lecture
- Affiche un message de succès si la lecture réussit
- **Ferme le fichier dans tous les cas (avec finally)**
- Retourne le contenu ou None en cas d'erreur

```
def read_file(filename): # À vous de jouer ! pass # Tests # read_file("fichier_existant.txt") #
read_file("fichier_inexistant.txt")
```

💡 **Indice:** Utilisez le bloc `finally` pour fermer le fichier, ou mieux: utilisez `with` .

Exercice 4: Accès à un dictionnaire

Récupérer une valeur de dictionnaire en toute sécurité

Facile

Écrivez une fonction `safe_dict_access(dct, key)` qui :

- Accède à une clé du dictionnaire
- Capture l'exception `KeyError` si la clé n'existe pas
- Affiche un message approprié
- Retourne la valeur ou None

```
def safe_dict_access(dct, key): # À vous de jouer ! pass # Tests user = {"name": "Alice", "age": 30} print(safe_dict_access(user, "name")) # Attendu: Alice print(safe_dict_access(user, "email")) # Attendu: None + message
```

💡 **Indice:** L'exception KeyError se lève quand on accède à une clé inexistante.

Exercice 5: Validation de données

Valider et convertir une entrée utilisateur

Moyen

Écrivez une fonction `validate_age(age_str)` qui :

- Convertit la chaîne en entier (capture `ValueError`)
- Vérifie que l'âge est entre 0 et 150 (sinon, raise une exception personnalisée)
- Affiche un message de succès si tout est OK
- Retourne l'âge valide ou None en cas d'erreur

```
def validate_age(age_str): try: age = int(age_str) if age < 0 or age > 150: raise ValueError("L'âge doit être entre 0 et 150") # Code de succès ici except ValueError as e: print(f"❌ Erreur: {e}") return None # À compléter avec else et/ou return # Tests print(validate_age("25")) # Attendu: 25 + message de succès print(validate_age("abc")) # Attendu: None + message d'erreur print(validate_age("200")) # Attendu: None + message d'erreur
```

💡 **Indice:** Utilisez un bloc `else` pour retourner l'âge si tout va bien. Vous pouvez aussi lever une exception avec `raise ValueError(...)` .

Exercice 6: Gestion d'API (simulation)

Simuler un appel API avec gestion d'erreurs

Difficile

Écrivez une fonction `fetch_user_data(user_id)` qui :

- Valide que user_id est un entier positif (raise ValueError sinon)
- Appelle une fonction `api_call(user_id)` qui peut lever :
  - `ConnectionError` (API indisponible)
  - `ValueError` (user_id invalide)
- Capture chaque type d'erreur avec un message approprié
- Affiche un message de succès dans le cas heureux
- Loggue toujours (même avec erreur) dans un bloc finally

```
# Simulation de l'API def api_call(user_id): import random if user_id < 0: raise ValueError("ID utilisateur invalide") if random.random() < 0.3: # 30% de chance de timeout raise ConnectionError("L'API est indisponible") return {"id": user_id, "name": f"User_{user_id}"} def fetch_user_data(user_id): try: # À vous de jouer ! data = api_call(user_id) except ConnectionError: # Gerer ici pass except ValueError as
```

```
e: # Gérer ici pass else: # Code de succès pass finally: # Toujours logger pass # Tests fetch_user_data(1) fetch_user_data(-1)
```

> 💡 **Indice:** Mettez du code de logging simple dans le finally (ex: print("Requête traitée")).

Conseils pour les exercices

- Testez vos fonctions avec des cas normaux ET des cas d'erreur
- Affichez des messages clairs pour que l'utilisateur comprenne ce qui s'est passé
- Utilisez les blocs `else` et `finally` pour structurer votre code
- Ciblez les exceptions spécifiques, ne faites pas `except:`
- En production, loggez toujours les erreurs!